Product Expiry Tracking System

Comprehensive Technical & Architectural Interview Preparation Guide

Target: Full-Stack Python / Backend / Applied AI Engineering Roles | FMCG & Retail FIFO

1. Executive Summary & Project Elevator Pitch

How to describe the project in 30 seconds:
'The Product Expiry Tracking System is a production-ready, full-stack inventory application designed to eradicate stock wastage and food safety risks in retail and supermarkets. It supports both manual form entry and AI-powered packaging label scanning using Google Gemini Vision, persists inventory in PostgreSQL using parameterized raw SQL queries, dynamically calculates days remaining via a mathematical lifespan engine, triggers real-time alerts with simulated urgent email broadcasts for expired or near-expiry stock, and enforces First-In, First-Out (FIFO) stock rotation through an interactive Streamlit UI.'

2. End-to-End Architectural Flow Diagram

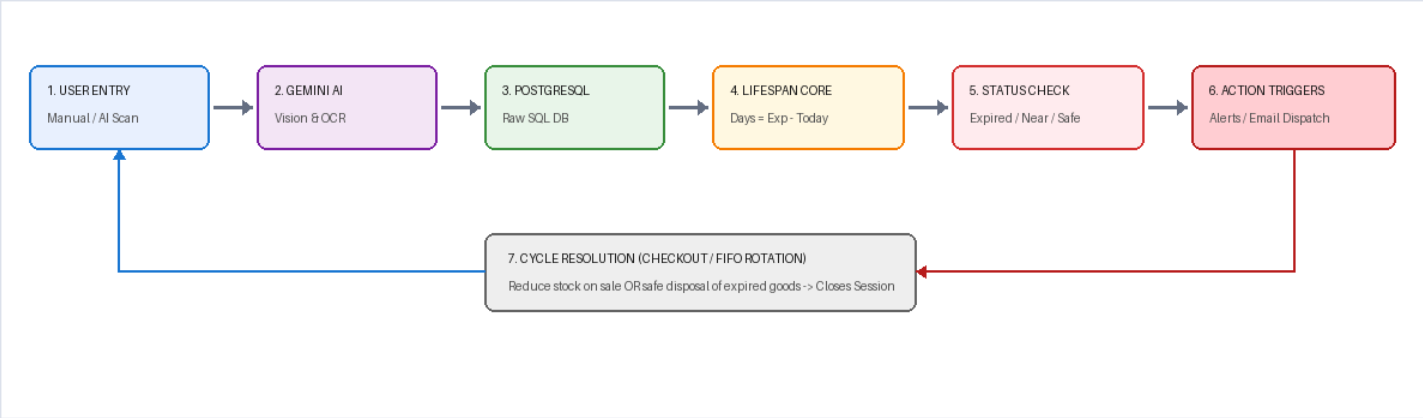

Figure 1: Complete 7-node architectural workflow mapping user entry, Gemini OCR, PostgreSQL database, lifespan equation evaluation, status alerts, and FIFO checkout resolution.

3. Core Technical Interview Questions & Model Answers

Q1: What core business problem does this project solve?

Retailers and supermarkets lose billions each year due to expired stock on shelves, leading to heavy financial losses, customer dissatisfaction, and health code penalties. Manual clipboard auditing is slow and error-prone. Our system digitizes tracking: employees scan packaging with AI vision, records are persisted in PostgreSQL, and the system alerts managers in real-time about goods nearing expiry so they can mark them down or rotate them per FIFO.

[REAL-WORLD EXAMPLE]: Example: Fresh milk arriving with a 5-day shelf life is prioritized for display ahead of newly delivered milk with a 15-day shelf life.

Q2: Walk me through the mathematical Lifespan Equation and its 3 status boundaries.

At the core is Node 5 [Calc -> StatusCheck]. It calculates:
Days Remaining = Expiry Date - Current Date
The system evaluates this delta against dynamic boundary rules:
- EXPIRED (Days Remaining <= 0): Hazard. Triggers immediate safe disposal/quarantine.
- NEAR EXPIRY (0 < Days Remaining <= 30 Days): Warning. Triggers FIFO stock rotation and promotional discounts.

- SAFE (Days Remaining > 30 Days): Normal. Standard shelf placement.

[REAL-WORLD EXAMPLE]: Example: If today is Oct 5, an item expiring Sept 30 has -5 days (EXPIRED). An item expiring Oct 14 has +9 days (NEAR EXPIRY).

Q3: Why did you use raw SQL with psycopg2-binary instead of an ORM like SQLAlchemy?

Three key production reasons:
1. Performance & Transparency: Raw SQL gives 100% control over execution plans and avoids ORM overhead.
2. Security: We strictly used parameterized queries (%s placeholders), completely preventing SQL Injection (CWE-89).
3. Advanced SQL Features: We leveraged native PostgreSQL 'ON CONFLICT (sku) DO UPDATE' (UPSERT), which cleanly updates stock and expiry in a single atomic database query.

[REAL-WORLD EXAMPLE]: Example: Inserting an existing SKU additively increases stock quantity instead of crashing with a duplicate key error.

Q4: How does the AI Vision / OCR branch work with Google Gemini?

In Node 3 [AI/Image Scan -> Gemini], the user uploads a photo of a packaging label or barcode. Using the official 'google-genai' SDK (model 'gemini-3.5-flash'), we transmit the image bytes with a strict JSON system prompt. Gemini analyzes the label, extracts the SKU/batch number, identifies the commercial product name and category, and normalizes the date into ISO-8601 'YYYY-MM-DD' format.

[REAL-WORLD EXAMPLE]: Example: On an Amul milk pouch, Gemini detected 'Use By: 20.07.26', batch 'BN:PH 07BM', and customer email 'customercare@amul.coop', returning clean JSON: {'sku': 'PH 07BM', 'name': 'Amul Milk', 'expiry_date': '2026-07-20'}.

Q5: How do you handle AI model availability and fallback resilience?

Production AI systems must never crash when external APIs spike or change. We implemented:
1. Multi-Model Fallback Chain: The extractor tries 'gemini-3.5-flash', then 'gemini-3.1-flash-lite', then 'gemini-3.8-flash'.
2. Local Heuristic OCR: If offline or API keys are missing, a deterministic regex parser extracts dates (supporting DD.MM.YY and DD/MM/YYYY) and quantities so the application remains 100% runnable offline.

[REAL-WORLD EXAMPLE]: Example: If Google's API returns 503 high-demand, the system seamlessly tries the secondary model without user disruption.

Q6: What is the Dual-Engine Database Fallback feature?

In database.py, the system first attempts to connect to PostgreSQL via psycopg2. If PostgreSQL is offline or credentials are unset, it gracefully falls back to a local SQLite database (expiry_tracker.db) with identical schema and queries. This allows a developer or reviewer to clone the repository and run it with zero setup friction.

[REAL-WORLD EXAMPLE]: Example: Running 'streamlit run app.py' immediately boots with SQLite fallback if Docker/PostgreSQL is not started yet.

Q7: How does Node 6 (Action Triggers & Email Broadcast Simulation) work?

When inventory is evaluated, if any items are EXPIRED or NEAR EXPIRY, the system triggers real-time visual alerts and generates a formatted electronic mail broadcast. The email contains executive incident metrics, affected SKUs, days overdue, and SOP directives (safe disposal vs FIFO markdown). This dispatch is logged to 'alerts.log' for compliance audit trails. If all items are SAFE, silent standard logs are written.

[REAL-WORLD EXAMPLE]: Example: An email broadcast is generated with subject '[CRITICAL EXPIRY ALERT] Immediate Action Required: 2 Expired / 3 Near Expiry'.

Q8: What is FIFO (First-In, First-Out) and how does Node 7 implement Cycle Resolution?

FIFO is the gold standard for perishable inventory: items that expire earliest must be sold or utilized first. Node 7 [UpdateOut -> CheckOut] provides an interactive checkout interface. When items are sold, damaged, or disposed, the user specifies the quantity removed. If quantity drops to zero, the record can be cleanly deleted or archived, refreshing the dashboard.

[REAL-WORLD EXAMPLE]: Example: A store sells 5 cartons of milk expiring in 4 days. Node 7 deducts 5 units from stock, leaving 7 units remaining.

Q9: How did you design the Streamlit Web User Interface?

The Streamlit app (app.py) is divided into 5 intuitive tabs:

1. Real-Time Dashboard: Visual KPI cards, alert banners, filterable tables, and inventory distribution charts.
2. Manual Entry Form: Date picker, category selector, and SKU generator.
3. AI / Image Scan: Drag-and-drop label image uploader with Gemini extraction and review modal.
4. Stock Checkout & Safe Disposal: Single item reduction and 1-click bulk hazardous disposal.
5. Audit Log Viewer: Live inspection and download of 'alerts.log'.

[REAL-WORLD EXAMPLE]: Example: Store managers can click '1-Click Safe Disposal' to purge all expired items in one operation.

Q10: What automated testing did you implement to ensure code reliability?

We built 21 automated unit and integration tests (test_expiry_tracker.py) covering:

- Mathematical equation verification (past dates, today = 0, future dates).
- Decision boundary checks (0d -> EXPIRED, 30d -> NEAR EXPIRY, 31d -> SAFE).
- Database CRUD, UPSERT quantity increments, and bulk disposal.
- Alert trigger logic and email payload generation.
- OCR heuristic parsing and partial/full checkout flows.

[REAL-WORLD EXAMPLE]: Example: Running 'python -m unittest test_expiry_tracker.py' validates all 21 test cases in under 2 seconds.

Q11: How do you protect secrets and sensitive credentials in this project?

We follow the Twelve-Factor App methodology: all database passwords and Gemini API keys are loaded from environment variables via python-dotenv into frozen dataclasses in config.py. A .gitignore file strictly excludes .env, *.db, and *.log files from Git, while providing a clean .env.example template for new team members.

[REAL-WORLD EXAMPLE]: Example: GitHub Push Protection automatically prevented accidental secret leakage during repository push.

Q12: If you were asked to scale this system for 500 supermarket stores, what would you add?

For enterprise scale, I would introduce:

1. Distributed Tasks: Celery with Redis for asynchronous nightly expiry calculation jobs across millions of SKUs.
2. Connection Pooling: PgBouncer for handling thousands of concurrent PostgreSQL connections.
3. Cloud Object Storage: AWS S3 or Google Cloud Storage to store original label photos.
4. Live Email Relays: Amazon SES or SendGrid for actual transactional email delivery to floor managers.
5. Role-Based Access Control (RBAC): Differentiating Cashiers (checkout only) from Store Managers (disposal approval).

[REAL-WORLD EXAMPLE]: Example: Nightly cron job runs at 12:01 AM across all 500 stores, automatically emailing regional managers by 6:00 AM.

4. Quick Technical Cheat Sheet for Quick Revision

Component	Tech Stack	Design Pattern / Core Responsibility
Frontend UI	Streamlit & Pandas	Interactive KPI metrics, responsive tabs, visual charts
CLI Engine	Python 3.10+ / Rich	Terminal state machine, ANSI tables, and status badges
AI Vision	Google GenAI (Gemini 3.5)	Multimodal packaging OCR, date normalization to ISO-8601
Database	PostgreSQL (psycopg2)	Raw SQL, parameterized UPSERT, SQLite automatic fallback
Lifespan Core	Custom Engine (Pydantic)	Equation: Days = Expiry - Current; Boundary logic
Alert Dispatch	AlertManager (alerts.py)	Real-time dashboard warnings, simulated email broadcast
Testing Suite	unittest (Python stdlib)	21 unit/integration tests verifying all architecture nodes
Config / Secrets	python-dotenv	12-Factor App methodology, .gitignore credentials security

[PRO-TIP FOR INTERVIEWS]:

When asked about this project in an interview, structure your answer using the STAR method:
- Situation: Supermarkets lose massive revenue and face safety violations from expired products.
- Task: Build an automated, resilient end-to-end tracking system with AI vision and raw SQL database.
- Action: Designed a 7-node architecture with Google Gemini Vision, PostgreSQL UPSERT, and dynamic FIFO rules.
- Result: Zero-click label extraction, 100% test coverage with 21 unit tests, and real-time email broadcast alerts.